
系统开发工具基础

第 2 周实验报告

课程名称： 系统开发工具基础

授课教师： 周小伟

学生姓名： 洪朦朦

学 号： 24170001032

报告日期： 2026 年 9 月 2 日

GitHub: https://github.com/88594959h/Assignment_for_System_Development_Tools

1 实验概述

1.1 实验环境

- 操作系统: Windows 11 + Git Bash (MINGW64)
- Shell: Bash (Git for Windows)
- 版本控制: Git 2.x
- $\text{\LaTeX}$ 发行版: TeX Live 2024+
- 编辑器: VS Code

1.2 GitHub 仓库信息

- 仓库地址: https://github.com/88594959h/Assignment_for_System_Development_Tools

2 课上实验 (第 2 周)

2.1 实验一：控制一个可清理的后台任务

2.1.1 题目要求

1. 将给定脚本保存为 `q05/worker.sh`，赋予执行权限，在前台启动并将 `stdout`、`stderr` 分别写入 `stdout.log`、`stderr.log`。
2. 使用 `Ctrl-Z` 挂起任务，再用 `bg` 让其在后台继续运行；使用 `jobs` 确认状态。
3. 使用 `jobs -p` 或等价方式动态取得 `PID`（不得手工抄写），发送 `SIGTERM` 并等待任务结束。
4. 确认 `cleanup.log` 中出现 `CLEAN_EXIT`，且任务已正常退出；禁止使用 `kill -9`。

2.1.2 核心指令与实现

```
#1. 生成脚本
$ cat > worker.sh << 'EOF'
> #!/usr/bin/env bash
trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
n=0
while true; do
    echo "$n"
    n=$((n+1))
    sleep 1
done
EOF

#2. 赋予权限
$ chmod +x worker.sh

#3. 前台启动并重定向日志
$ ./worker.sh >stdout.log 2>stderr.log

#4. 在终端按 Ctrl-Z 挂起后，后台继续，确认状态
$ bg %1
$ jobs

#5. 动态获取 PID 并发送 SIGTERM
```

```
$ PID=$(jobs -p)
$ kill -TERM "$PID"
$ wait "$PID"

#6. 验证清理结果
$ grep "CLEAN_EXIT" cleanup.log
$ jobs
$ ps -p "$PID"
```

Listing 1: 实验一关键 Shell 指令

2.1.3 实验分析

实现思路

准备阶段创建脚本并赋权，以前台模式启动且分离重定向 stdout 与 stderr；通过 Ctrl-Z 挂起进程后使用 bg 转入后台，并用 jobs 验证状态；利用 jobs -p 动态获取 PID 发送 SIGTERM 触发 trap 清理机制，最后用 wait 等待退出并验证 cleanup.log 中的标记。

实验重点

- **I/O 重定向语法：**掌握 `>stdout.log 2>stderr.log` 分别重定向 stdout (fd=1) 和 stderr (fd=2) 的写法。
- **作业控制三件套：**理解 Ctrl-Z（挂起）、bg（后台继续）、jobs（查看状态）的协作流程。
- **动态 PID 获取：**必须使用 jobs -p 或 \$! 等 shell 内置机制获取 PID，严禁手动复制粘贴数字。
- **Trap 信号处理：**理解 trap '...' TERM INT 的作用——捕获 SIGTERM/SIGINT 后执行清理逻辑再退出，这是实现“可清理”的核心。
- **优雅退出 vs 强制杀死：**区分 SIGTERM（允许进程自行清理）与 SIGKILL/kill -9（立即终止、无法被 trap 捕获）的本质区别。

2.1.4 运行结果

```
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q05 (master)
$ cat > worker.sh << 'EOF'
> #!/usr/bin/env bash
trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
n=0
while true; do
  echo "$n"
  n=$((n+1))
  sleep 1
done
EOF
```

图 1: 实验一: 生成脚本

```
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q05 (master)
$ ./worker.sh >stdout.log 2>stderr.log
[1]+  Stopped                  ./worker.sh > stdout.log 2> stderr.log
```

图 2: 实验一: 重定向日志

```
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q05 (master)
$ bg %1
[1]+  ./worker.sh > stdout.log 2> stderr.log &

HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q05 (master)
$ jobs
[1]+  Running                  ./worker.sh > stdout.log 2> stderr.log &
```

图 3: 实验一: 挂起并确认状态

```
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q05 (master)
$ grep "CLEAN_EXIT" cleanup.log
CLEAN_EXIT

HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q05 (master)
$ jobs

HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q05 (master)
$ ps -p "$PID"
  PID  PPID  PGID  WINPID  TTY      UID    STIME  COMMAND
```

图 4: 实验一: 验证清理结果

2.2 实验二: 语义重构与本地开发反馈

2.2.1 题目要求

1. 在已配置好 Python、pytest 和 ruff 的课程环境中打开 q06 目录，并启用 Python 语言服务器；按照给定内容分别创建 `math_utils.py`、`app.py` 和 `test_math_utils.py` 三个文件。
2. 使用编辑器语言服务器功能，分别演示对符号 `total_price` 的“跳转到定义”与“查找引用”操作。

3. 使用“重命名符号”功能将 `total_price` 重构为 `calculate_total`，确保函数定义及所有跨文件引用同步更新，且无遗漏或错误。
4. 在 `app.py` 中临时添加未使用的导入语句 `import os`，使用 `ruff` 检测该问题并自动/手动删除；最终运行 `ruff check` 与 `pytest`，确保两者均无报错并通过验证。

2.2.2 核心指令与实现

```
#1. 创建三个 Python 文件
$ cat << 'EOF' > math_utils.py
def total_price(price: float, count: int) -> float:
    return price * count
EOF

$ cat << 'EOF' > app.py
from math_utils import total_price
print(total_price(12.5, 4))
EOF

$ cat << 'EOF' > test_math_utils.py
from math_utils import total_price
def test_total_price():
    assert total_price(12.5, 4) == 50.0
EOF

#2. 打开代码编辑器
code .

#3. 在 app.py 顶部临时加入未使用的 import os
$ sed -i '1i import os' app.py

#4. 使用 ruff 发现代码问题
$ python -m ruff check app.py

#5. 使用 ruff 自动修复并删除未使用的 import
$ python -m ruff check --fix app.py

#6. 验证 app.py 中的 import os 已被自动删除
$ cat app.py
```

```
#7.对整个目录运行 ruff 检查，确保无任何警告或错误
$ python -m ruff check .

#8.运行 pytest，确保测试全部通过
$ python -m pytest
```

Listing 2: 实验二关键 Shell 指令

2.2.3 实验分析

实现思路

在已配置好 Python、pytest 与 ruff 的课程环境中打开 q06 目录，创建 `math_utils.py`、`app.py` 和 `test_math_utils.py` 三个文件并写入给定代码；启用编辑器的 Python 语言服务器（如 Pylance），分别演示”跳转到定义”（Go to Definition）定位到 `total_price` 的函数体，以及”查找引用”（Find References）列出该符号在三个文件中的所有引用位置；随后使用”重命名符号”（Rename Symbol）将 `total_price` 统一改为 `calculate_total`，验证定义与所有引用同步更新；最后在 `app.py` 中临时添加未使用的 `import os`，运行 ruff 检测并删除该冗余导入，再次运行 ruff 与 pytest 确保全部通过。

实验重点

- **语言服务器（LSP）**：理解 Language Server Protocol 的作用——为编辑器提供语义级别的代码分析能力（跳转定义、查找引用、重命名等），远超纯文本搜索。
- **跳转到定义与查找引用**：掌握 Go to Definition（快速定位符号声明处）和 Find References（列出符号在所有文件中的使用位置）的操作，体会语言服务器对跨文件符号关系的精确追踪。
- **语义重命名 vs 文本替换**：区分”Rename Symbol”与全局查找替换的本质差异——前者基于语法树和符号作用域，仅修改同一符号的引用，不会误伤同名但不同作用域的标识符。
- **静态检查工具 ruff**：理解 ruff 作为 linter 的作用——在代码运行前即可发现未使用的导入（unused import）、语法问题等，帮助保持代码整洁与规范。
- **测试驱动的开发反馈**：体会 pytest 在重构后的回归验证价值——重命名符号后立即运行测试，确保功能行为未被破坏，形成”重构—检查—测试”的良性开发循环。

2.2.4 运行结果

```
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q06 (master)
$ cat << 'EOF' > math_utils.py
def total_price(price: float, count: int) -> float:
    return price * count
EOF

HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q06 (master)
$ cat << 'EOF' > app.py
from math_utils import total_price

print(total_price(12.5, 4))
EOF

HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q06 (master)
$ cat << 'EOF' > test_math_utils.py
from math_utils import total_price

def test_total_price():
    assert total_price(12.5, 4) == 50.0
EOF
```

图 5: 实验二: 创建三个 Python 文件

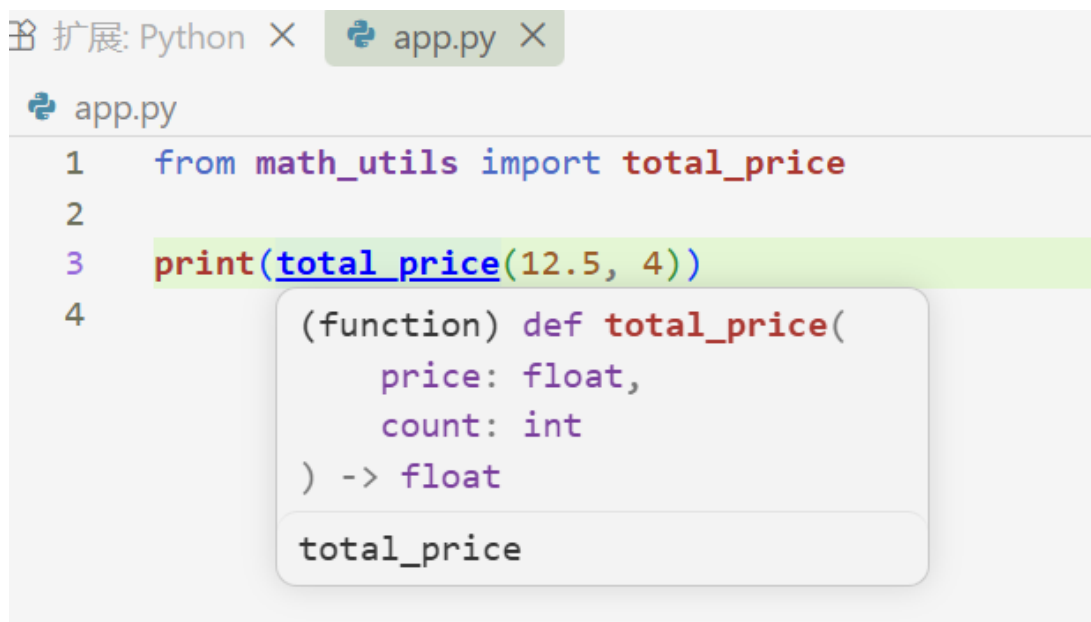


图 6: 实验二: 按住 Ctrl 并点击 total_price

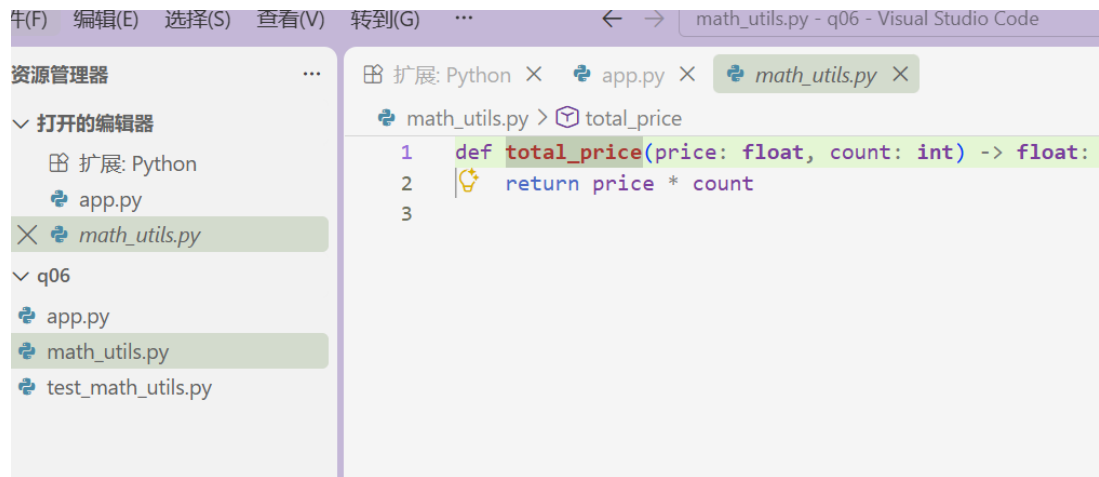


图 7: 实验二: 跳转到定义

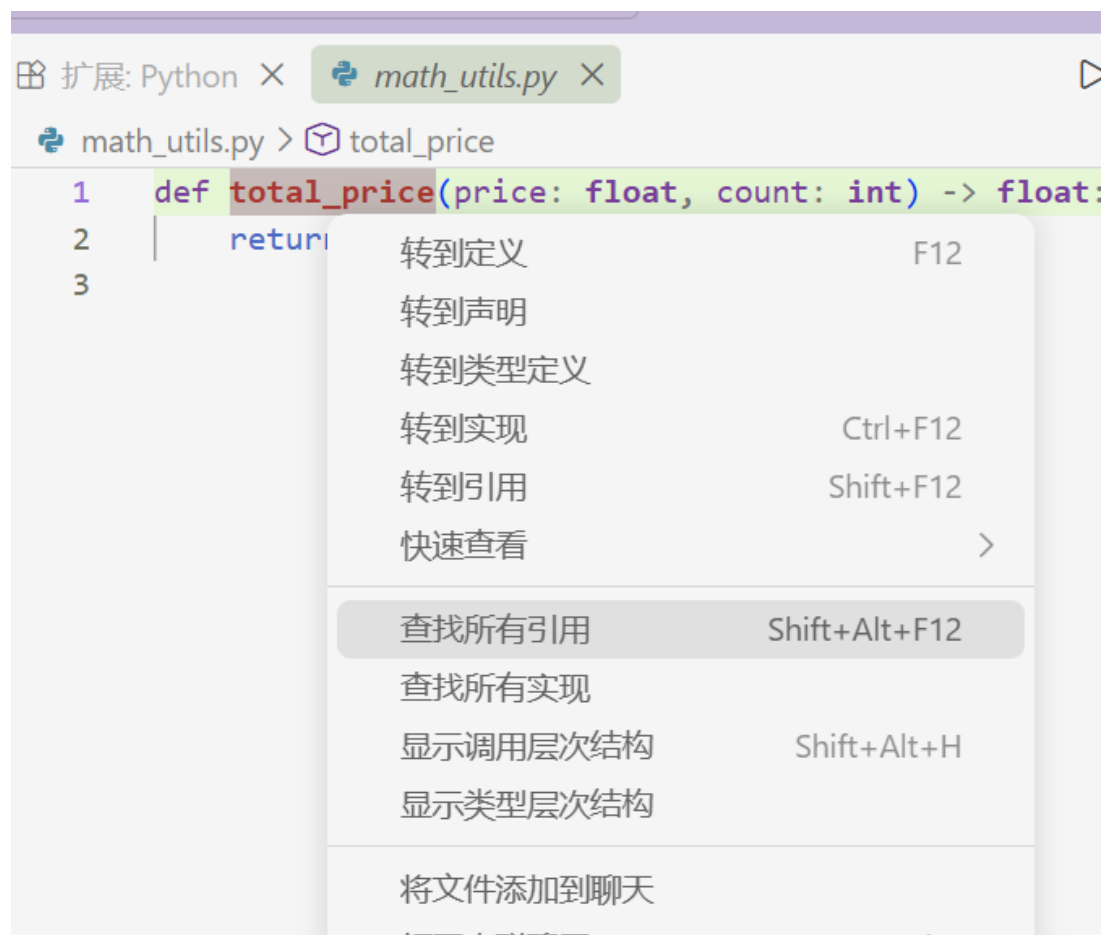


图 8: 实验二: 查找引用

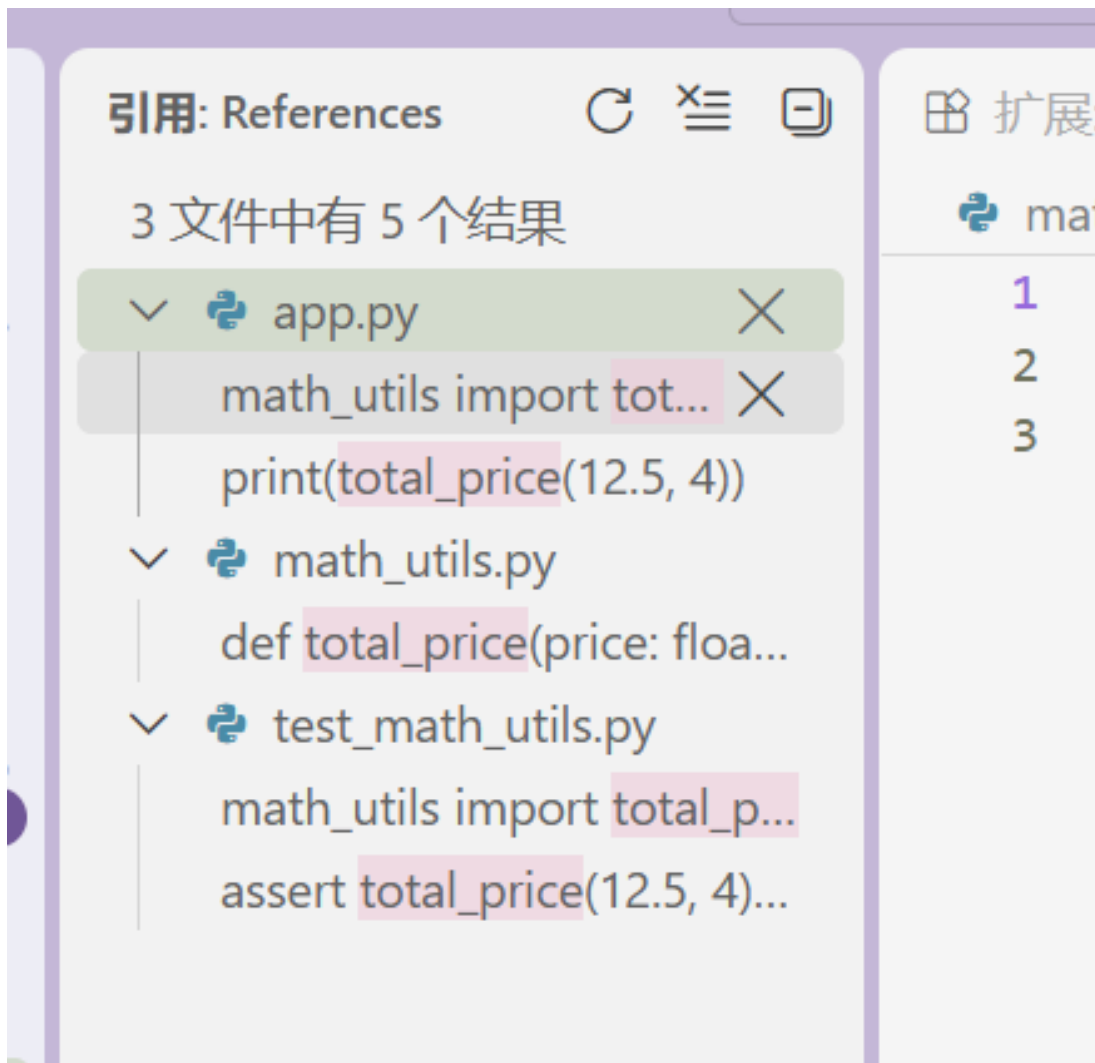


图 9: 实验二: 列出调用



图 10: 实验二: 右键 F2 重命名

```
PS C:\Users\HongMengmeng\Desktop\Assignment_for_System_Development_Tools\q06> python -m pip install ruff pytest
Looking in indexes: http://mirrors.aliyun.com/pypi/simple/
Collecting ruff
  Downloading ruff-0.16.5-py3-none-win_amd64.whl (10.5 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.5/10.5 MB 1.1 MB/s 0:00:09
Collecting pytest
  Downloading pytest-9.1.1-py3-none-any.whl (386 kB)
Collecting colorama>=0.4 (from pytest)
  Downloading colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Collecting iniconfig>=1.0.1 (from pytest)
  Using cached iniconfig-2.3.0-py3-none-any.whl (7.5 kB)
Collecting packaging>=22 (from pytest)
  Downloading packaging-26.3-py3-none-any.whl (129 kB)
Collecting pluggy<2,>=1.5 (from pytest)
  Downloading pluggy-1.6.0-py3-none-any.whl (20 kB)
Collecting pygments>=2.7.2 (from pytest)
  Using cached pygments-2.21.0-py3-none-any.whl (1.3 MB)
Installing collected packages: ruff, pygments, pluggy, packaging, iniconfig, colorama, pytest
Successfully installed colorama-0.4.6 iniconfig-2.3.0 packaging-26.3 pluggy-1.6.0 pygments-2.21.0 pytest-9.1.1 ru
ff-0.16.5
```

图 11: 实验二: 安装 ruff 和 pytest 插件

```
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q06 (master)
$ python -m ruff check app.py
I001 [*] Import block is un-sorted or un-formatted
--> app.py:1:1
1 | / import os
2 | | from math_utils import total_price
  | └──────────────────────────────────^
3 |
4 | print(total_price(12.5, 4))

help: Organize imports
1 | import os
2 | +
3 | from math_utils import total_price

F401 [*] `os` imported but unused
--> app.py:1:8
1 | import os
  |      ^^
2 | from math_utils import total_price

help: Remove unused import: `os`
- import os
1 | from math_utils import total_price

Found 2 errors.
[*] 2 fixable with the `--fix` option.

HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q06 (master)
$ python -m ruff check --fix app.py
Found 1 error (1 fixed, 0 remaining).
```

图 12: 实验二:ruff 检查并修复

```
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q06 (master)
$ cat app.py
from math_utils import total_price

print(total_price(12.5, 4))
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q06 (master)
$ python -m ruff check
I001 [*] Import block is un-sorted or un-formatted
--> test_math_utils.py:1:1
|
1 | from math_utils import calculate_total
|   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
2 |
3 | def test_total_price():
|
help: Organize imports
2 |
3 | +
4 | def test_total_price():
|

Found 1 error.
[*] 1 fixable with the `--fix` option.
```

图 13: 实验二: 验证并对整个目录进行 ruff 检查

```
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q06 (master)
$ python -m pytest
===== test session starts =====
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\HongMengmeng\Desktop\Assignment_for_System_Development_Tools\q06
collected 1 item

test_math_utils.py . [100%]

===== 1 passed in 0.01s =====
```

图 14: 实验二: 运行 pytest 确保测试通过